

RA

**Ref1 for STUDENT RECORD DB:**

**"Database Design and Implementation"**

Edward Sciore, Boston College

ISBN: 978-0-471-75716-0

**Ref2 for COMPANY DB:**

**"Fund. of Database Systems",**

Elmasri, Navathe, 5th ed., Addison Wesley

# RA:relational algebra

- has a similar power with SQL.
- RA has NO formal syntax while SQL is industry standard language.
- RA is task oriented query language(procedural) while SQL is result-oriented (declarative).
- DBMS translate SQL to RA in order to execute it.
- RA query can be expressed as a *query tree* that contains nodes for tables and operators within the query. *query tree* shows execution order and is used in execution&optimization.

## operators

- Single-table operators:

- Select:  $\sigma$

- Project:  $\pi$

- Sort  $S$

- Rename  $\rho$

- Extend  $E$

- Aggregate functions:

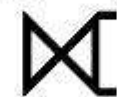
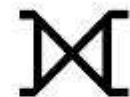
$\mathcal{F}$  <kümeleme fonksiyon listesi>

- Groupby:

<gruplama nitelikleri>  $\mathcal{F}$  <kümeleme fonksiyon listesi>

- 2-table operators

- Union:
- intersection:
- set difference:
- division:
- product:
- theta-join :
- natural join:
- semijoin:
- antijoin:
- full outerjoin:
- left outer join:
- right outer join:



# Seçim İşlemi (Select Operation)

- Notasyon:  $\sigma_p(r)$
- $p$  secim kriteri (selection predicate)
- Tanım:  $\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$

*p burada propositional calculustaki bir formüldür ve  $\wedge$  (and),  $\vee$  (or),  $\neg$  (not) terimlerini bağlanmış şekilde içermektedir.*

Her terim

**<attribute> op <attribute> veya <constant>**

**ki op: =,  $\neq$ , >,  $\geq$ , <,  $\leq$  olmaktadır.**

# Seçim İşlemi (Select Operation) – Örnek

*R ilişkisi*

| <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> |
|----------|----------|----------|----------|
| $\alpha$ | $\alpha$ | 1        | 7        |
| $\alpha$ | $\beta$  | 5        | 7        |
| $\beta$  | $\beta$  | 12       | 3        |
| $\beta$  | $\beta$  | 23       | 10       |

■  $\sigma_{A=B \text{ and } D > 5}(r)$

| <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> |
|----------|----------|----------|----------|
| $\alpha$ | $\alpha$ | 1        | 7        |
| $\beta$  | $\beta$  | 23       | 10       |

# İzdüşüm-Atma işlemi (Project Operation)

- Notasyon:  $\Pi_{A_1, A_2, \dots, A_k}(r)$   
 $A_1, A_2$  öznelik adları ve  $r$  ilişki adı.
- Sonuç listelenmemiş olan kolonların silinmesinden sonra ortaya çıkan  $k$  kolonlu ilişkidir. ilişkiler set olduğundan, çoklu satırlar sonuçtan silinir.

| <i>A</i> | <i>B</i> | <i>C</i> |
|----------|----------|----------|
| $\alpha$ | 10       | 1        |
| $\alpha$ | 20       | 1        |
| $\beta$  | 30       | 1        |
| $\beta$  | 40       | 2        |

$\Pi_{A,C}(r)$

| <i>A</i> | <i>C</i> |
|----------|----------|
| $\alpha$ | 1        |
| $\beta$  | 1        |
| $\beta$  | 2        |



# Birleştirme İşlemi (Union Operation)

- Notasyon:  $r \cup s$
- Tanım:  $r \cup s = \{t \mid t \in r \text{ or } t \in s\}$
- $r \cup s$  nun geçerli olması için.
  1.  $r, s$  aynı sayıda özniteliğe sahip olmalıdır (arity)
  2. öznitelik alanları uyumlu (compatible) olmalıdır (örnek:  $r$ 'nin 2. kolonu  $s$ 'nin 2. kolonu ile aynı tip değerlere sahip olması)

| <i>A</i> | <i>B</i> |
|----------|----------|
| $\alpha$ | 1        |
| $\alpha$ | 2        |
| $\beta$  | 1        |

*r*

| <i>A</i> | <i>B</i> |
|----------|----------|
| $\alpha$ | 2        |
| $\beta$  | 3        |

*s*

$r \cup s$ :

| <i>A</i> | <i>B</i> |
|----------|----------|
| $\alpha$ | 1        |
| $\alpha$ | 2        |
| $\beta$  | 1        |
| $\beta$  | 3        |



# Küme Kesişimi İşlemi (Set-Intersection Operation)

- Notasyon:  $r \cap s$
- Tanım:  $r \cap s = \{ t \mid t \in r \text{ and } t \in s \}$
- Varsayım:
  - $r$  ve  $s$  aynı arity'e sahip
  - $r$  ve  $s$  öznitellikleri uyumlu
- Not:  $r \cap s = r - (r - s)$

| A        | B |
|----------|---|
| $\alpha$ | 1 |
| $\alpha$ | 2 |
| $\beta$  | 1 |

$r$

| A        | B |
|----------|---|
| $\alpha$ | 2 |
| $\beta$  | 3 |

$s$

$r \cap s$

| A        | B |
|----------|---|
| $\alpha$ | 2 |

# Fark İşlemi (Set Difference Operation)

- Notasyon  $r - s$
- tanım:  $r - s = \{t \mid t \in r \text{ and } t \notin s\}$

Fark İşlemi uyumlu (compatible) ilişkiler içersinde yapılır.

- $r$  ve  $s$  aynı arity ye sahip olmalı
- $r$  ve  $s$  nin öznitelik alanları uyumlu olmalı.

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\alpha$ | 2   |
| $\beta$  | 1   |

$r$

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 2   |
| $\beta$  | 3   |

$s$

$r - s:$

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\beta$  | 1   |

# renaming

Notation:  $\rho_{\text{OldName} \rightarrow \text{NewName}}(r)$

## Paternity

| Father  | Child   |
|---------|---------|
| Adam    | Cain    |
| Adam    | Abel    |
| Abraham | Isaac   |
| Abraham | Ishmael |

## Maternity

| Mother | Child   |
|--------|---------|
| Eve    | Cain    |
| Eve    | Seth    |
| Sarah  | Isaac   |
| Hagar  | Ishmael |

$\rho_{\text{Father} \rightarrow \text{Parent}}(\text{Paternity}) \cup \rho_{\text{Mother} \rightarrow \text{Parent}}(\text{Maternity})$

| Parent  | Child   |
|---------|---------|
| Adam    | Cain    |
| Adam    | Abel    |
| Abraham | Isaac   |
| Abraham | Ishmael |
| Eve     | Cain    |
| Eve     | Seth    |
| Sarah   | Isaac   |
| Hagar   | Ishmael |

# Kartezyen Çarpım İşlemi (Cartesian-Product Operation)

- Notasyon  $r \times s$
- Tanım:  $r \times s = \{t \ q \mid t \in r \text{ and } q \in s\}$

| <i>A</i> | <i>B</i> |
|----------|----------|
| $\alpha$ | 1        |
| $\beta$  | 2        |

*r*

| <i>C</i> | <i>D</i> | <i>E</i> |
|----------|----------|----------|
| $\alpha$ | 10       | <i>a</i> |
| $\beta$  | 10       | <i>a</i> |
| $\beta$  | 20       | <i>b</i> |
| $\gamma$ | 10       | <i>b</i> |

*s*

$r \times s$ :

| <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> | <i>E</i> |
|----------|----------|----------|----------|----------|
| $\alpha$ | 1        | $\alpha$ | 10       | <i>a</i> |
| $\alpha$ | 1        | $\beta$  | 10       | <i>a</i> |
| $\alpha$ | 1        | $\beta$  | 20       | <i>b</i> |
| $\alpha$ | 1        | $\gamma$ | 10       | <i>b</i> |
| $\beta$  | 2        | $\alpha$ | 10       | <i>a</i> |
| $\beta$  | 2        | $\beta$  | 10       | <i>a</i> |
| $\beta$  | 2        | $\beta$  | 20       | <i>b</i> |
| $\beta$  | 2        | $\gamma$ | 10       | <i>b</i> |

# Theta-JOIN ( $\bowtie_p$ )

**theta-join**: a derived operator

$$r_1 \bowtie_p r_2 = \sigma_p(r_1 \times r_2)$$

if  $p$  is a conjunction of equalities, then we have an **equi-join**

## Employees

| Employee | Project |
|----------|---------|
| Smith    | A       |
| Black    | A       |
| Black    | B       |

## Projects

| Code | Name  |
|------|-------|
| A    | Venus |
| B    | Mars  |

## Employees $\bowtie_{\text{Project=Code}}$ Projects

| Employee | Project | Code | Name  |
|----------|---------|------|-------|
| Smith    | A       | A    | Venus |
| Black    | A       | A    | Venus |
| Black    | B       | B    | Mars  |

# NATURAL JOIN ( \* veya $\bowtie$ )

$r_1$

| Employee | Department |
|----------|------------|
| Smith    | sales      |
| Black    | production |
| White    | production |

$r_2$

| Department | Head  |
|------------|-------|
| production | Mori  |
| sales      | Brown |

$r_1 * r_2$  ( $r_1 \bowtie r_2$ )

| Employee | Department | Head  |
|----------|------------|-------|
| Smith    | sales      | Brown |
| Black    | production | Mori  |
| White    | production | Mori  |

# left, right, full outer JOIN

**r<sub>1</sub>**

| Employee | Department |
|----------|------------|
| Smith    | sales      |
| Black    | production |
| White    | production |

**r<sub>2</sub>**

| Department | Head  |
|------------|-------|
| production | Mori  |
| purchasing | Brown |

**r<sub>1</sub>**

⋈

**r<sub>2</sub>**

| Employee | Department | Head |
|----------|------------|------|
| Smith    | Sales      | NULL |
| Black    | production | Mori |
| White    | production | Mori |

**r<sub>1</sub>**

⋈

**r<sub>2</sub>**

| Employee | Department | Head  |
|----------|------------|-------|
| Black    | production | Mori  |
| White    | production | Mori  |
| NULL     | purchasing | Brown |

**r<sub>1</sub>**

⋈

**r<sub>2</sub>**

| Employee | Department | Head  |
|----------|------------|-------|
| Smith    | Sales      | NULL  |
| Black    | production | Mori  |
| White    | production | Mori  |
| NULL     | purchasing | Brown |



# Semijoin ( $R \bowtie S$ )

$$R \bowtie S = \pi_{a_1, \dots, a_n}(R * S).$$

$a_1, \dots, a_n$  is a set of attribute names of  $R$ . Thus, The result of semijoin is the set of all tuples in  $R$  for which there is a tuple in  $S$  that is equal on their common attribute names.

**Employee**

| Name    | EmpId | Dept Name  |
|---------|-------|------------|
| Harry   | 3415  | Finance    |
| Sally   | 2241  | Sales      |
| George  | 3401  | Finance    |
| Harriet | 2202  | Production |

**Dept**

| Dept Name  | Manager |
|------------|---------|
| Sales      | Bob     |
| Sales      | Thomas  |
| Production | Katie   |
| Production | Mark    |

**Employee  $\bowtie$  Dept**

| Name    | EmpId | DeptName   |
|---------|-------|------------|
| Sally   | 2241  | Sales      |
| Harriet | 2202  | Production |

# Antijoin ( $R \triangleright S$ )

The antijoin, is similar to the semijoin, but the result of an antijoin is only those tuples in  $R$  for which there is no tuple in  $S$  that is equal on their common attribute names. Antijoin CAN be simulated by a JOIN.

Let it be "c"

$$A \triangleright B = \pi_{\sigma} (A \bowtie B)$$

$A$ : attr.     $B$ : c: null     $A$  c:  $B$ : c

Employee

| Name    | EmpId | Dept Name  |
|---------|-------|------------|
| Harry   | 3415  | Finance    |
| Sally   | 2241  | Sales      |
| George  | 3401  | Finance    |
| Harriet | 2202  | Production |

Dept

| Dept Name  | Manager |
|------------|---------|
| Sales      | Sally   |
| Production | Harriet |

Employee  $\triangleright$  Dept

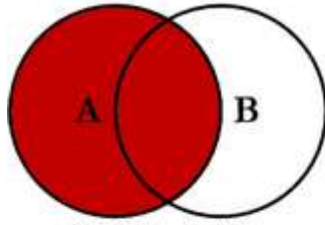
| Name   | EmpId | DeptName |
|--------|-------|----------|
| Harry  | 3415  | Finance  |
| George | 3401  | Finance  |

# SQL JOINS

B ile birleşen VE  
birleşmeyen A'lar

~~A ∩ B~~

~~A?~~

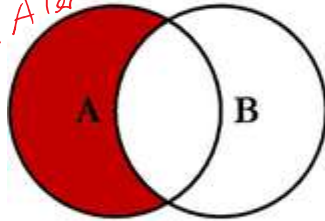


```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```

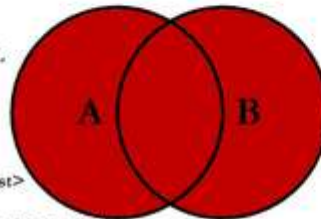
B ile birleşmeyen A'lar

~~A ∩ B~~

~~A - B?~~



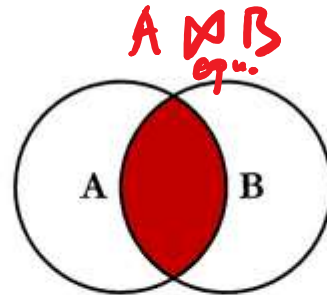
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```

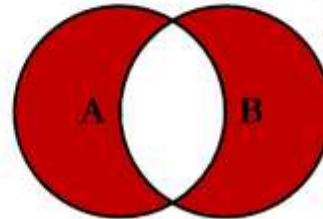
~~A ∩ B~~

~~A ∪ B?~~



```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```

© C.L. Moffatt, 2003



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

~~(A ∩ B) ∪ (B ∩ A)~~

~~A ∩ B?~~

A ile birleşen VE  
birleşmeyen B'ler

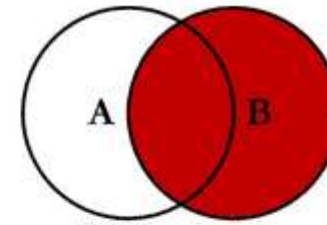
~~A ∩ B~~

~~B?~~

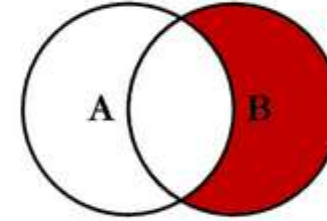
A ile birleşmeyen B'ler

~~B ∩ A~~

~~B - A?~~



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```

<https://www.codeproject.com/Articles/33052/Visual-Representation-of-SQL-Joins>

# DIVISION

- **DIVISION Operation**

- The division operation is applied to two relations
- $R(Z) \div S(X)$ , where  $X$  subset  $Z$ . Let  $Y = Z - X$  (and hence  $Z = X \cup Y$ ); that is, let  $Y$  be the set of attributes of  $R$  that are not attributes of  $S$ .
- The result of DIVISION is a relation  $T(Y)$  that includes a tuple  $t$  if tuples  $t_R$  appear in  $R$  with  $t_R[Y] = t$ , and with
  - $t_R[X] = t_s$  for every tuple  $t_s$  in  $S$ .
- For a tuple  $t$  to appear in the result  $T$  of the DIVISION, the values in  $t$  must appear in  $R$  in combination with every tuple in  $S$ .

# example

(a)

SSN\_PNOS

| Essn      | Pno |
|-----------|-----|
| 123456789 | 1   |
| 123456789 | 2   |
| 666884444 | 3   |
| 453453453 | 1   |
| 453453453 | 2   |
| 333445555 | 2   |
| 333445555 | 3   |
| 333445555 | 10  |
| 333445555 | 20  |
| 999887777 | 30  |
| 999887777 | 10  |
| 987987987 | 10  |
| 987987987 | 30  |
| 987654321 | 30  |
| 987654321 | 20  |
| 888665555 | 20  |

SMITH\_PNOS

| Pno |
|-----|
| 1   |
| 2   |

SSNS

| Ssn       |
|-----------|
| 123456789 |
| 453453453 |

(b)

R

| A  | B  |
|----|----|
| a1 | b1 |
| a2 | b1 |
| a3 | b1 |
| a4 | b1 |
| a1 | b2 |
| a3 | b2 |
| a2 | b3 |
| a3 | b3 |
| a4 | b3 |
| a1 | b4 |
| a2 | b4 |
| a3 | b4 |

S

| A  |
|----|
| a1 |
| a2 |
| a3 |

T

| B  |
|----|
| b1 |
| b4 |

R nin öyle satırları ki; R[Y] kısmındaki bir değer bütün S[X] ler ile beraber gözüksün.

**Figure 6.8**

The DIVISION operation. (a) Dividing SSN\_PNOS by SMITH\_PNOS. (b)  $T \leftarrow R \div S$ .

$$R \div S \equiv \begin{cases} T_1 = \pi_Y(R) \\ T_2 = \pi_Y[(S \times T_1) - R] \\ T_3 = T_1 - T_2 \end{cases}$$



# Kümeleme (*aggregate*) fonksiyonları

- A type of request that cannot be expressed in the basic relational algebra is to specify mathematical **aggregate functions** on collections of values from the database.
- Examples of such functions include retrieving the average or total salary of all employees or the total number of employee tuples.
  - These functions are used in simple statistical queries that summarize information from the database tuples.
- Common functions applied to collections of numeric values include
  - SUM, AVERAGE, MAXIMUM, and MINIMUM.
- The COUNT function is used for counting tuples or values.

# Kümeleme (*aggregate*) fonksiyonları

- Use of the Aggregate Functional operation  $\mathcal{F}$ 
  - $\mathcal{F}_{\text{MAX Salary}}(\text{EMPLOYEE})$  retrieves the maximum salary value from the EMPLOYEE relation
  - $\mathcal{F}_{\text{MIN Salary}}(\text{EMPLOYEE})$  retrieves the minimum Salary value from the EMPLOYEE relation
  - $\mathcal{F}_{\text{SUM Salary}}(\text{EMPLOYEE})$  retrieves the sum of the Salary from the EMPLOYEE relation
  - $\mathcal{F}_{\text{COUNT SSN, AVERAGE Salary}}(\text{EMPLOYEE})$  computes the count (number) of employees and their average salary
    - Note: count just counts the number of rows, without removing duplicates

# Grouping with Aggregation

- The previous examples all summarized one or more attributes for a set of tuples
  - Maximum Salary or Count (number of) Ssn
- Grouping can be combined with Aggregate Functions
- Example: For each department, retrieve the DNO, COUNT SSN, and AVERAGE SALARY
- A variation of aggregate operation  $\mathcal{F}$  allows this:
  - Grouping attribute placed to left of symbol
  - Aggregate functions to right of symbol
  - $\text{DNO } \mathcal{F} \text{ COUNT SSN, AVERAGE Salary (EMPLOYEE)}$
- Above operation groups employees by DNO (department number) and computes the count of employees and average salary per department



# example

**Figure 6.10**

The aggregate function operation.

(a)  $\rho_{R(Dno, No\_of\_employees, Average\_sal)} (Dno \int COUNT Ssn, AVERAGE Salary (EMPLOYEE)).$

(b)  $Dno \int COUNT Ssn, AVERAGE Salary (EMPLOYEE).$

(c)  $\int COUNT Ssn, AVERAGE Salary (EMPLOYEE).$  *→ start from here!*

**R**

(a)

| Dno | No_of_employees | Average_sal |
|-----|-----------------|-------------|
| 5   | 4               | 33250       |
| 4   | 3               | 31000       |
| 1   | 1               | 55000       |

(b)

| Dno | Count_ssn | Average_salary |
|-----|-----------|----------------|
| 5   | 4         | 33250          |
| 4   | 3         | 31000          |
| 1   | 1         | 55000          |

(c)

| Count_ssn | Average_salary |
|-----------|----------------|
| 8         | 35125          |

/

**Figure 8.6**

Results of GROUP BY and HAVING. (a) Q24. (b) Q26.

(a)

| Fname    | Minit | Lname   | Ssn       | ... | Salary | Super_ssn | Dno |
|----------|-------|---------|-----------|-----|--------|-----------|-----|
| John     | B     | Smith   | 123456789 |     | 30000  | 333445555 | 5   |
| Franklin | T     | Wong    | 333445555 |     | 40000  | 888665555 | 5   |
| Ramesh   | K     | Narayan | 666884444 |     | 38000  | 333445555 | 5   |
| Joyce    | A     | English | 453453453 | ... | 25000  | 333445555 | 5   |
| Alicia   | J     | Zelaya  | 999887777 |     | 25000  | 987654321 | 4   |
| Jennifer | S     | Wallace | 987654321 |     | 43000  | 888665555 | 4   |
| Ahmad    | V     | Jabbar  | 987987987 |     | 25000  | 987654321 | 4   |
| James    | E     | Bong    | 888665555 |     | 55000  | NULL      | 1   |

Result of Q24

| Dno | Count (*) | Avg (Salary) |
|-----|-----------|--------------|
| 5   | 4         | 33250        |
| 4   | 3         | 31000        |
| 1   | 1         | 55000        |

Grouping EMPLOYEE tuples by the value of Dno

# Properties of algebra:

## Complete Set of operations

- The set of operations including SELECT  $\sigma$ , PROJECT  $\pi$ , UNION  $\cup$ , DIFFERENCE  $-$ , RENAME  $\rho$ , and CARTESIAN PRODUCT  $\times$  is called a *complete set* because any other relational algebra expression can be expressed by a combination of these five operations.
- For example:
  - $R \cap S = (R \cup S) - ((R - S) \cup (S - R))$
  - $R \bowtie_{\langle \text{join condition} \rangle} S = \sigma_{\langle \text{join condition} \rangle} (R \times S)$

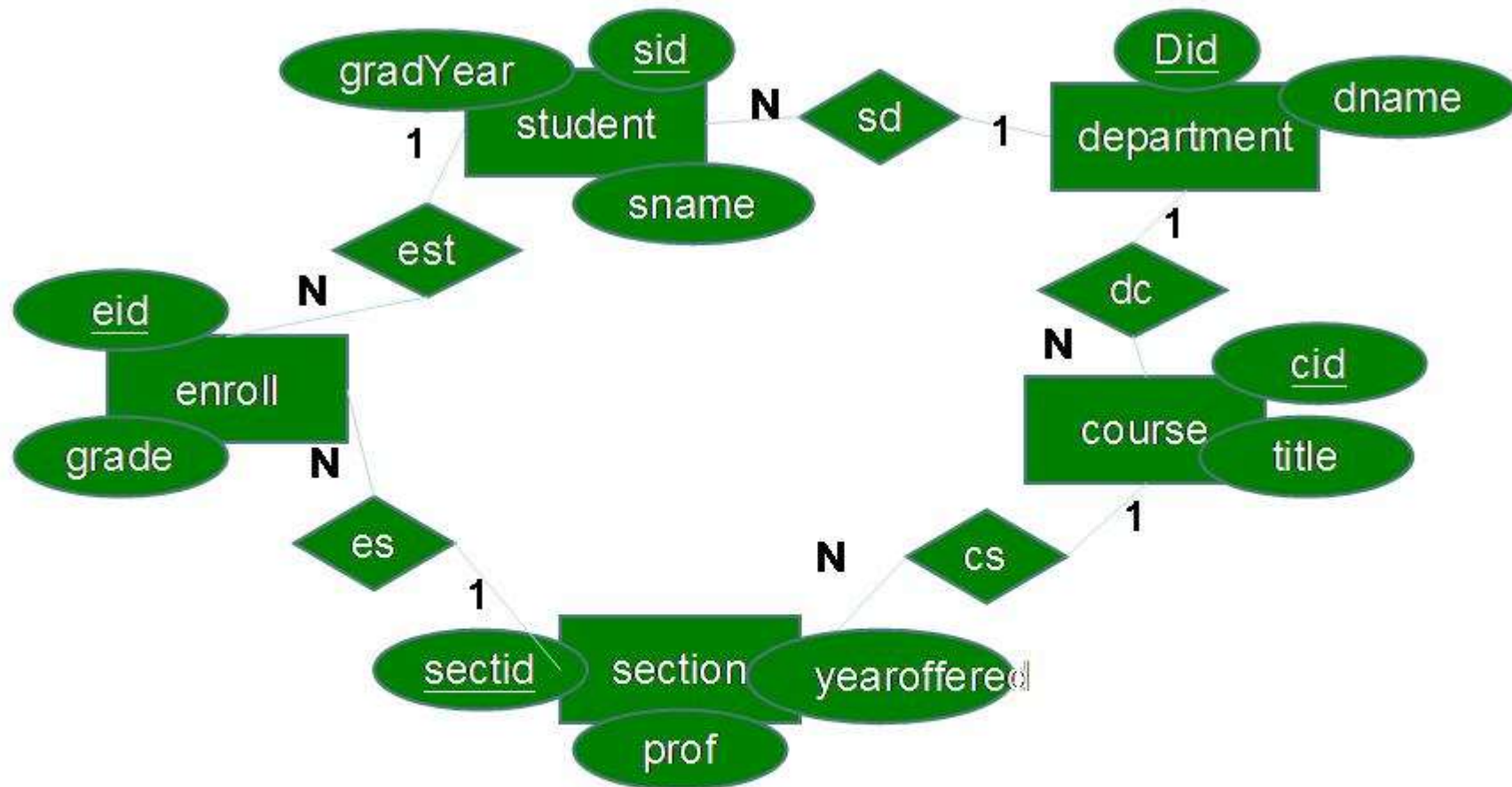
# Properties of algebra:

## Prop. of UNION, INTERSECTION, MINUS

- Notice that both union and intersection are commutative operations; that is
- $R \cup S = S \cup R$ , and  $R \cap S = S \cap R$
- Both union and intersection can be treated as n-ary operations applicable to any number of relations as both are associative operations; that is
- $R \cup (S \cup T) = (R \cup S) \cup T$
- $(R \cap S) \cap T = R \cap (S \cap T)$
- The minus operation is not commutative; that is, in general
- $R - S \neq S - R$
- All (plus cartesian and JOIN) constitutes Complete set of operations
- Example:  $R \cap S = (R \cup S) - ((R - S) \cup (S - R))$



# A student record database:



```
STUDENT(SId, SName, GradYear, MajorId)
DEPT(DId, DName)
COURSE(CId, Title, DeptId)
SECTION(SectId, CourseId, Prof, YearOffered)
ENROLL(EId, StudentId, SectionId, Grade)
```

# Student record db state

| STUDENT | SId | SName | GradYear | MajorId |
|---------|-----|-------|----------|---------|
|         | 1   | joe   | 2004     | 10      |
|         | 2   | amy   | 2004     | 20      |
|         | 3   | max   | 2005     | 10      |
|         | 4   | sue   | 2005     | 20      |
|         | 5   | bob   | 2003     | 30      |
|         | 6   | kim   | 2001     | 20      |
|         | 7   | art   | 2004     | 30      |
|         | 8   | pat   | 2001     | 20      |
|         | 9   | lee   | 2004     | 10      |

| DEPT | DId | DName   |
|------|-----|---------|
|      | 10  | compsci |
|      | 20  | math    |
|      | 30  | drama   |

| COURSE | CId | Title      | DeptId |
|--------|-----|------------|--------|
|        | 12  | db systems | 10     |
|        | 22  | compilers  | 10     |
|        | 32  | calculus   | 20     |
|        | 42  | algebra    | 20     |
|        | 52  | acting     | 30     |
|        | 62  | elocution  | 30     |

| SECTION | SectId | CourseId | Prof     | YearOffered |
|---------|--------|----------|----------|-------------|
|         | 13     | 12       | turing   | 2004        |
|         | 23     | 12       | turing   | 2005        |
|         | 33     | 32       | newton   | 2000        |
|         | 43     | 32       | einstein | 2001        |
|         | 53     | 62       | brando   | 2001        |

| ENROLL | EId | StudentId | SectionId | Grade |
|--------|-----|-----------|-----------|-------|
|        | 14  | 1         | 13        | A     |
|        | 24  | 1         | 43        | C     |
|        | 34  | 2         | 43        | B+    |
|        | 44  | 4         | 33        | B     |
|        | 54  | 4         | 53        | A     |
|        | 64  | 6         | 53        | A     |

**Figure 1-1**  
Some records for a university database

# select

STUDENT(SId, SName, GradYear, MajorId)  
DEPT(DId, DName)  
COURSE(CId, Title, DeptId)  
SECTION(SectId, CourseId, Prof, YearOffered)  
ENROLL(EId, StudentId, SectionId, Grade)

- list students who graduated in 2004:
  - Q1:  $\sigma_{\text{GradYear}=2004}(\text{STUDENT})$
- list students who graduated in 2004 with a major of 10 or 20:
  - Q2:  $\sigma_{\text{GradYear}=2004 \text{ and } (\text{MajorId}=10 \text{ or } \text{MajorId}=20)}(\text{STUDENT})$
  - Q3:  $\sigma_{\text{MajorId}=10 \text{ or } \text{MajorId}=20}(\sigma_{\text{GradYear}=2004}(\text{STUDENT}))$



# project

STUDENT(SId, SName, GradYear, MajorId)  
DEPT(DId, DName)  
COURSE(CId, Title, DeptId)  
SECTION(SectId, CourseId, Prof, YearOffered)  
ENROLL(EId, StudentId, SectionId, Grade)

- List the name and graduation year of all students:
  - Q5:  $\pi_{SName, GradYear}(STUDENT)$
- List the name of all students having major 10
  - Q6:  $\pi_{SName}(\sigma_{MajorId=10}(STUDENT))$
- What about the following query?
  - Q7:  $\sigma_{MajorId=10}(\pi_{SName}(STUDENT))$

## sort, rename, extend

STUDENT(SId, SName, GradYear, MajorId)  
DEPT(DId, DName)  
COURSE(CId, Title, DeptId)  
SECTION(SectId, CourseId, Prof, YearOffered)  
ENROLL(EId, StudentId, SectionId, Grade)

- Sort the students by graduation year and then name
  - Q8:  $\sigma_{\text{GradYear}, \text{SName}}(\text{STUDENT})$ 
    - *Sort operator is typically the outmost operator.*
- Rename the records from Q6 with field *SName* as *CSMajors*
  - Q9:  $\rho_{\text{SName}, \text{CSMajors}}(\text{Q6})$ 
    - *rename operator is used in multi-table operators (s.a. Product, join) in order to alter schema of one table to match the other's.*
- Extend STUDENT table to have the calculated field *GradClass* as the the number years since 1863 (first grad year)
  - Q10:  $E_{\text{GradYear}-1863, \text{GradClass}}(\text{STUDENT})$
- Extend STUDENT table to have the calculated field *College* as "YTU"
  - Q11:  $E_{\text{YTU}, \text{College}}(\text{STUDENT})$

# Group by

STUDENT(SId, SName, GradYear, MajorId)  
 DEPT(DId, DName)  
 COURSE(CId, Title, DeptId)  
 SECTION(SectId, CourseId, Prof, YearOffered)  
 ENROLL(EId, StudentId, SectionId, Grade)

| STUDENT | SId | SName | GradYear | MajorId |
|---------|-----|-------|----------|---------|
|         | 1   | joe   | 2004     | 10      |
|         | 2   | amy   | 2004     | 20      |
|         | 3   | max   | 2005     | 10      |
|         | 4   | sue   | 2005     | 20      |
|         | 5   | bob   | 2003     | 30      |
|         | 6   | kim   | 2001     | 20      |
|         | 7   | art   | 2004     | 30      |
|         | 8   | pat   | 2001     | 20      |
|         | 9   | lee   | 2004     | 10      |

| Q12 | MajorId | MinOfGradYear | MaxOfGradYear |
|-----|---------|---------------|---------------|
|     | 10      | 2004          | 2005          |
|     | 20      | 2001          | 2005          |
|     | 30      | 2003          | 2004          |

- Show max and min graduation year per major
  - Q12:  $\text{MajorId } F_{\text{Min(GradYear), Max(GradYear)}}(\text{STUDENT})$
- Show the number of students in each major per graduation year.
  - Q13:  $\text{MajorId, GradYear } F_{\text{Count(SId)}}(\text{STUDENT})$

| Q13 | MajorId | GradYear | CountOfSId |
|-----|---------|----------|------------|
|     | 10      | 2004     | 2          |
|     | 10      | 2005     | 1          |
|     | 20      | 2001     | 2          |
|     | 20      | 2004     | 1          |
|     | 20      | 2005     | 1          |
|     | 30      | 2003     | 1          |
|     | 30      | 2004     | 1          |



# Group by

STUDENT(SId, SName, GradYear, MajorId)  
DEPT(DId, DName)  
COURSE(CId, Title, DeptId)  
SECTION(SectId, CourseId, Prof, YearOffered)  
ENROLL(EId, StudentId, SectionId, Grade)

| STUDENT | SId | SName | GradYear | MajorId |
|---------|-----|-------|----------|---------|
|         | 1   | joe   | 2004     | 10      |
|         | 2   | amy   | 2004     | 20      |
|         | 3   | max   | 2005     | 10      |
|         | 4   | sue   | 2005     | 20      |
|         | 5   | bob   | 2003     | 30      |
|         | 6   | kim   | 2001     | 20      |
|         | 7   | art   | 2004     | 30      |
|         | 8   | pat   | 2001     | 20      |
|         | 9   | lee   | 2004     | 10      |

- **Show the min graduation year of any student**
  - Q14:  $F_{\text{Min(GradYear)}} ( \text{STUDENT} )$
- **Show the list of major ids of all students with duplicates removed.**
  - Q15:  $\text{MajorId } F ( \text{STUDENT} )$
- **Show the number of students having a major.**
  - Q16:  $F_{\text{Count(MajorId)}} ( \text{STUDENT} )$ 
    - *Count function count all records including duplicates. Count function ignores NULL*
- **Show the number of different majors**
  - Q17:  $F_{\text{CountDistinct(MajorId)}} ( \text{STUDENT} )$ 
    - *CountDistinct (and other aggregate funcs.) ignores NULL*

# Group by

STUDENT(SId, SName, GradYear, MajorId)  
 DEPT(DId, DName)  
 COURSE(CId, Title, DeptId)  
 SECTION(SectId, CourseId, Prof, YearOffered)  
 ENROLL(EId, StudentId, SectionId, Grade)

| ENROLL | EId | StudentId | SectionId | Grade |
|--------|-----|-----------|-----------|-------|
|        | 14  | 1         | 13        | A     |
|        | 24  | 1         | 43        | C     |
|        | 34  | 2         | 43        | B+    |
|        | 44  | 4         | 33        | B     |
|        | 54  | 4         | 53        | A     |
|        | 64  | 6         | 53        | A     |

| Q18 | EId | StudentId | SectionId | Grade |
|-----|-----|-----------|-----------|-------|
|     | 14  | 1         | 13        | A     |
|     | 54  | 4         | 53        | A     |
|     | 64  | 6         | 53        | A     |

| Q19 | SectionId | CountOfEId |
|-----|-----------|------------|
|     | 13        | 1          |
|     | 53        | 2          |

| Q20 | MaxOfCountOfEId |
|-----|-----------------|
|     | 2               |

- Show the most number of A's given in any section
  - 3 steps:
    - Q18:  $\sigma_{\text{Grade}='A'}(\text{ENROLL})$
    - Q19:  $\text{SectionId} \text{ } \overline{F}_{\text{Count}(\text{EId})}(\text{Q18})$
    - Q20:  $\overline{F}_{\text{Max}(\text{CountOfEId})}(\text{Q19})$



## union

- Requires both table has the same schema
- Find the combined names of students and professors

– Q52:  $\rho_{SName, Person}(\pi_{SName}(STUDENT))$

Q53:  $\rho_{Prof, Person}(\pi_{Prof}(SECTION))$

Q54:  $Q52 \cup Q53$

# product

STUDENT(SId, SName, GradYear, MajorId)  
 DEPT(DId, DName)  
 COURSE(CId, Title, DeptId)  
 SECTION(SectId, CourseId, Prof, YearOffered)  
 ENROLL(EId, StudentId, SectionId, Grade)

- All combinations of records from STUDENT and DEPT
  - Q22: (STUDENT  $\times$  DEPT)

| STUDENT | SId | SName | GradYear | MajorId |
|---------|-----|-------|----------|---------|
|         | 1   | joe   | 2004     | 10      |
|         | 2   | amy   | 2004     | 20      |
|         | 3   | max   | 2005     | 10      |
|         | 4   | sue   | 2005     | 20      |
|         | 5   | bob   | 2003     | 30      |
|         | 6   | kim   | 2001     | 20      |
|         | 7   | art   | 2004     | 30      |
|         | 8   | pat   | 2001     | 20      |
|         | 9   | lee   | 2004     | 10      |

| DEPT | DId | DName   |
|------|-----|---------|
|      | 10  | compsci |
|      | 20  | math    |
|      | 30  | drama   |

| Q22 | SId | SName | MajorId | GradYear | DId | DName   |
|-----|-----|-------|---------|----------|-----|---------|
|     | 1   | joe   | 10      | 2004     | 10  | compsci |
|     | 2   | amy   | 20      | 2004     | 10  | compsci |
|     | 3   | max   | 10      | 2005     | 10  | compsci |
|     | 4   | sue   | 20      | 2005     | 10  | compsci |
|     | 5   | bob   | 30      | 2003     | 10  | compsci |
|     | 6   | kim   | 20      | 2001     | 10  | compsci |
|     | 7   | art   | 30      | 2004     | 10  | compsci |
|     | 8   | pat   | 20      | 2001     | 10  | compsci |
|     | 9   | lee   | 10      | 2004     | 10  | compsci |
|     | 1   | joe   | 10      | 2004     | 20  | math    |
|     | 2   | amy   | 20      | 2004     | 20  | math    |
|     | 3   | max   | 10      | 2005     | 20  | math    |
|     | 4   | sue   | 20      | 2005     | 20  | math    |
|     | 5   | bob   | 30      | 2003     | 20  | math    |
|     | 6   | kim   | 20      | 2001     | 20  | math    |
|     | 7   | art   | 30      | 2004     | 20  | math    |
|     | 8   | pat   | 20      | 2001     | 20  | math    |
|     | 9   | lee   | 10      | 2004     | 20  | math    |
|     | 1   | joe   | 10      | 2004     | 30  | drama   |
|     | 2   | amy   | 20      | 2004     | 30  | drama   |
|     | 3   | max   | 10      | 2005     | 30  | drama   |
|     | 4   | sue   | 20      | 2005     | 30  | drama   |
|     | 5   | bob   | 30      | 2003     | 30  | drama   |
|     | 6   | kim   | 20      | 2001     | 30  | drama   |
|     | 7   | art   | 30      | 2004     | 30  | drama   |
|     | 8   | pat   | 20      | 2001     | 30  | drama   |
|     | 9   | lee   | 10      | 2004     | 30  | drama   |

- All students and their major department's name
  - Q23:  $\sigma_{\text{MajorId}=\text{DId}}(\text{STUDENT} \times \text{DEPT})$

# join

STUDENT(SId, SName, GradYear, MajorId)  
DEPT(DId, DName)  
COURSE(CId, Title, DeptId)  
SECTION(SectId, CourseId, Prof, YearOffered)  
ENROLL(EId, StudentId, SectionId, Grade)

- $T1 \bowtie_p T2 \equiv \sigma_p(T1 \times T2)$
- **All students and their major department's name**
  - Q24:  $(\text{STUDENT} \bowtie_{\text{MajorId=DId}} \text{DEPT})$
- **The grades Joe received during 2004**
  - Q25:  $\sigma_{\text{SName}='Joe'}(\text{STUDENT})$
  - Q26:  $(Q25 \bowtie_{\text{SId=StudentId}} \text{ENROLL})$
  - Q27:  $\sigma_{\text{YearOffered}=2004}(\text{SECTION})$
  - Q28:  $(Q26 \bowtie_{\text{SectId=SectionId}} Q27)$
  - Q29:  $\pi_{\text{Grade}}(Q28)$
- **(Q29') Find the grades Joe received during his graduation year.**
  - select YearOffered=GradYear, only after joining with SECTION
  - There are 3 predicates to join 3 tables, but only 2 is enough.
  - Named as circular query.

# Join

STUDENT(SId, SName, GradYear, MajorId)  
DEPT(DId, DName)  
COURSE(CId, Title, DeptId)  
SECTION(SectId, CourseId, Prof, YearOffered)  
ENROLL(EId, StudentId, SectionId, Grade)

- The sections that gave out the most A's.  
(remember Q20)
  - Q30:  $(Q20 \bowtie_{\text{MaxCountOfEId} = \text{CountOfEId}} Q19)$
  - Q31:  $(Q30 \bowtie_{\text{SectionId} = \text{SectId}} \text{SECTION})$
- List students that have the same major as Joe.
  - Q32:  $\pi_{\text{MajorId}}(\sigma_{\text{SName} = \text{'Joe'}}(\text{STUDENT}))$
  - Q33:  $\rho_{\text{MajorId}, \text{JoesMajor}}(Q32)$
  - Q34:  $(Q33 \bowtie_{\text{MajorId} = \text{JoesMajorId}} \text{STUDENT})$
  - This type of join is named as self-join (or recursive).
  - We have to rename in order to write predicate correctly.



STUDENT(SId, SName, GradYear, MajorId)  
 DEPT(DId, DName)  
 COURSE(CId, Title, DeptId)  
 SECTION(SectId, CourseId, Prof, YearOffered)  
 ENROLL(EId, StudentId, SectionId, Grade)

# semijoin

- Q24 lists pairs of matching records. Sometimes, we only need if there is match.
- List departments that have at least 1 student in their major.

– Q35:  $(\text{DEPT} \bowtie_{\text{DId=MajorId}} \text{STUDENT})$

- Q35 consists of those records from the first table that match some record in the second table.

– Equivalent join:

Q36:  $(\text{DEPT} \bowtie_{\text{DId=MajorId}} \text{STUDENT})$

Q37:  $\text{DId, DName} \text{ } \rho(\text{Q36})$

- Groupby operator projects and then removes duplicates..

- The students who took a course from Profesor Einstein.

– Q38:  $\sigma_{\text{Prof=Einstein}}(\text{SECTION})$

Q39:  $(\text{ENROLL} \bowtie_{\text{SectionId = SectId}} \text{Q38})$

Q40:  $(\text{STUDENT} \bowtie_{\text{SId=StudentId}} \text{Q39})$

$\rho(R \bowtie S) \Rightarrow \rho(R) \bowtie S$   
 R's str.

$\rho(R) \bowtie S \Rightarrow \rho(R \bowtie S)$

similar to  $\sigma_p(R)$

| SECTION | SectId | CourseId | Prof     | YearOffered |
|---------|--------|----------|----------|-------------|
|         | 13     | 12       | turing   | 2004        |
|         | 23     | 12       | turing   | 2005        |
|         | 33     | 32       | newton   | 2000        |
|         | 43     | 32       | einstein | 2001        |
|         | 53     | 62       | brando   | 2001        |

| ENROLL | EId | StudentId | SectionId | Grade |
|--------|-----|-----------|-----------|-------|
|        | 14  | 1         | 13        | A     |
|        | 24  | 1         | 43        | C     |
|        | 34  | 2         | 43        | B+    |
|        | 44  | 4         | 33        | B     |
|        | 54  | 4         | 53        | A     |
|        | 64  | 6         | 53        | A     |

| STUDENT | SId | SName | GradYear | MajorId |
|---------|-----|-------|----------|---------|
|         | 1   | joe   | 2004     | 10      |
|         | 2   | amy   | 2004     | 20      |
|         | 3   | max   | 2005     | 10      |
|         | 4   | sue   | 2005     | 20      |
|         | 5   | bob   | 2003     | 30      |
|         | 6   | kim   | 2001     | 20      |
|         | 7   | art   | 2004     | 30      |
|         | 8   | pst   | 2001     | 20      |
|         | 9   | lee   | 2004     | 10      |



STUDENT(SId, SName, GradYear, MajorId)  
 DEPT(DId, DName)  
 COURSE(CId, Title, DeptId)  
 SECTION(SectId, CourseId, Prof, YearOffered)  
 ENROLL(EId, StudentId, SectionId, Grade)

# antijoin

- Opposite of semijoin; thus returns exactly those records that the semijoin omits.
- List departments that have no students majors.
  - Q41: ( DEPT  $\bowtie_{DId=MajorId}$  STUDENT)
  - Note that union of Q35 and Q41 is DEPT and Q35 and Q41 has no common record.
- Find the sections where nobody received a grade of 'F'.
  - Q42:  $\sigma_{Grade \neq 'F'}(ENROLL)$
  - Q43: (SECTION  $\bowtie_{SectId = SectionId}$  Q42)
  - What about the following solution?
    - (  $\sigma_{Grade \neq 'F'}(ENROLL) \bowtie_{SectionId=SectId}$  SECTION )
- Antijoin is used in the following type of queries:
  - "NOT EXIST x" type  $\longrightarrow$  "-----  $\triangleright$  tuples qualifying x"
  - "FOR ALL x" = "NOT EXIST NOT x"  $\longrightarrow$  "-----  $\triangleright$  tuples that are NOT qualifying x"
- Find those sections in which everyone got an 'A'. (sections in which there does not exist someone who did not get an 'A')
  - $\sigma_{Grade \neq 'A'}(ENROLL)$
  - (SECTION  $\bowtie_{SectId = SectionId}$  Q42)

```

STUDENT(SId, SName, GradYear, MajorId)
DEPT(DId, DName)
COURSE(CId, Title, DeptId)
SECTION(SectId, CourseId, Prof, YearOffered)
ENROLL(EId, StudentId, SectionId, Grade)

```

# antijoin

- Find professors who have never given an 'F'.
  - Find Sections that have an 'F'.  
Exclude those sections' Profs from all professors.
  - Q44:  $\sigma_{Grade='F'}(ENROLL)$
  - Q45:  $(SECTION \bowtie_{SectId = SectionId} Q44)$
  - Q46:  $\rho_{Prof, BadProf}(Q45)$  // find sections where an 'F' was given.
  - Q47:  $(SECTION \bowtie_{Prof = BadProf} Q46)$
  - Q48:  $\rho_{Prof} F(Q47)$  //removes duplicates
- Find professors who gave at least one 'F' in every section they taught.
  - Find sections where nobody received a grade of 'F'. Q43 did this.  
Find those professors who did not give any section within Q43.
  - Q49:  $\rho_{Prof, GoodProf}(Q43)$
  - Q50:  $(SECTION \bowtie_{Prof = GoodProf} Q49)$
  - Q51:  $\rho_{Prof} F(Q50)$

hiç F verilmemiş olan section'ları vermemiş olan Prof'ları buluyoruz.

# outerjoin

STUDENT(SId, SName, GradYear, MajorId)  
 DEPT(DId, DName)  
 COURSE(CId, Title, DeptId)  
 SECTION(SectId, CourseId, Prof, YearOffered)  
 ENROLL(EId, StudentId, SectionId, Grade)

- Same arguments as the join
- Result table has the records of the join unioned with non-matching records from each of the tables.
- The outerjoin of STUDENT and ENROLL
  - Q55: (STUDENT  $\bowtie_{SId=StudentId}$  ENROLL)

| STUDENT | SId | SName | GradYear | MajorId |
|---------|-----|-------|----------|---------|
|         | 1   | joe   | 2004     | 10      |
|         | 2   | amy   | 2004     | 20      |
|         | 3   | max   | 2005     | 10      |
|         | 4   | sue   | 2005     | 20      |
|         | 5   | bob   | 2003     | 30      |
|         | 6   | kim   | 2001     | 20      |
|         | 7   | art   | 2004     | 30      |
|         | 8   | pat   | 2001     | 20      |
|         | 9   | lee   | 2004     | 10      |

| ENROLL | EId | StudentId | SectionId | Grade |
|--------|-----|-----------|-----------|-------|
|        | 14  | 1         | 13        | A     |
|        | 24  | 1         | 43        | C     |
|        | 34  | 2         | 43        | B+    |
|        | 44  | 4         | 33        | B     |
|        | 54  | 4         | 53        | A     |
|        | 64  | 6         | 53        | A     |

| Q55 | SId | SName | MajorId | GradYear | EId  | StudentId | SectionId | Grade |
|-----|-----|-------|---------|----------|------|-----------|-----------|-------|
|     | 1   | joe   | 10      | 2004     | 14   | 1         | 13        | A     |
|     | 1   | joe   | 10      | 2004     | 24   | 1         | 43        | C     |
|     | 2   | amy   | 20      | 2004     | 34   | 2         | 43        | B+    |
|     | 4   | sue   | 20      | 2005     | 44   | 4         | 33        | B     |
|     | 4   | sue   | 20      | 2005     | 54   | 4         | 53        | A     |
|     | 6   | kim   | 20      | 2001     | 64   | 6         | 53        | A     |
|     | 3   | max   | 10      | 2005     | null | null      | null      | null  |
|     | 5   | bob   | 30      | 2003     | null | null      | null      | null  |
|     | 7   | art   | 30      | 2004     | null | null      | null      | null  |
|     | 8   | pat   | 20      | 2001     | null | null      | null      | null  |
|     | 9   | lee   | 10      | 2004     | null | null      | null      | null  |



## outerjoin

```
STUDENT(SId, SName, GradYear, MajorId)
DEPT(DId, DName)
COURSE(CId, Title, DeptId)
SECTION(SectId, CourseId, Prof, YearOffered)
ENROLL(EId, StudentId, SectionId, Grade)
```

- Show the number of enrollments for each student that has taken at least one course
    - Q56:  $\text{StudentId } F_{\text{count}(EId)}(\text{ENROLL})$
  - Show the number of enrollments for all students
    - Q57:  $(\text{STUDENT} \bowtie_{SId=StudentId} \text{ENROLL})$
    - Q58:  $\text{SId } F_{\text{count}(EId)}(\text{Q57})$  // remember: count ignores NULL
- \ SId?

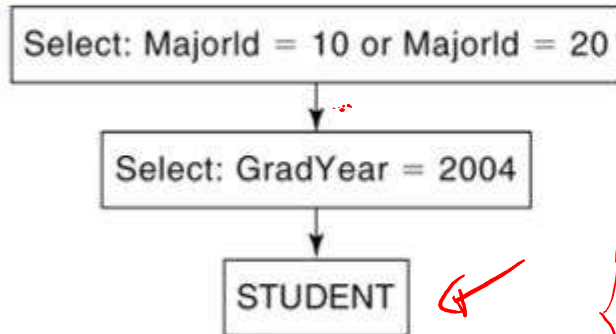
# Query Tree

- **Query Tree**
  - An internal data structure to represent a query
  - Standard technique for estimating the work involved in executing the query, the generation of intermediate results, and the optimization of execution
  - Nodes stand for operations like selection, projection, join, renaming, division, ....
  - Leaf nodes represent base relations
  - A tree gives a good visual feel of the complexity of the query and the operations involved
  - Algebraic Query Optimization consists of rewriting the query or modifying the query tree into an equivalent tree.

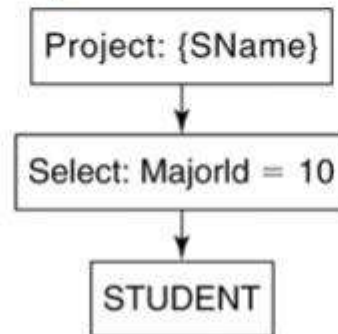


# Query Trees of previous examples

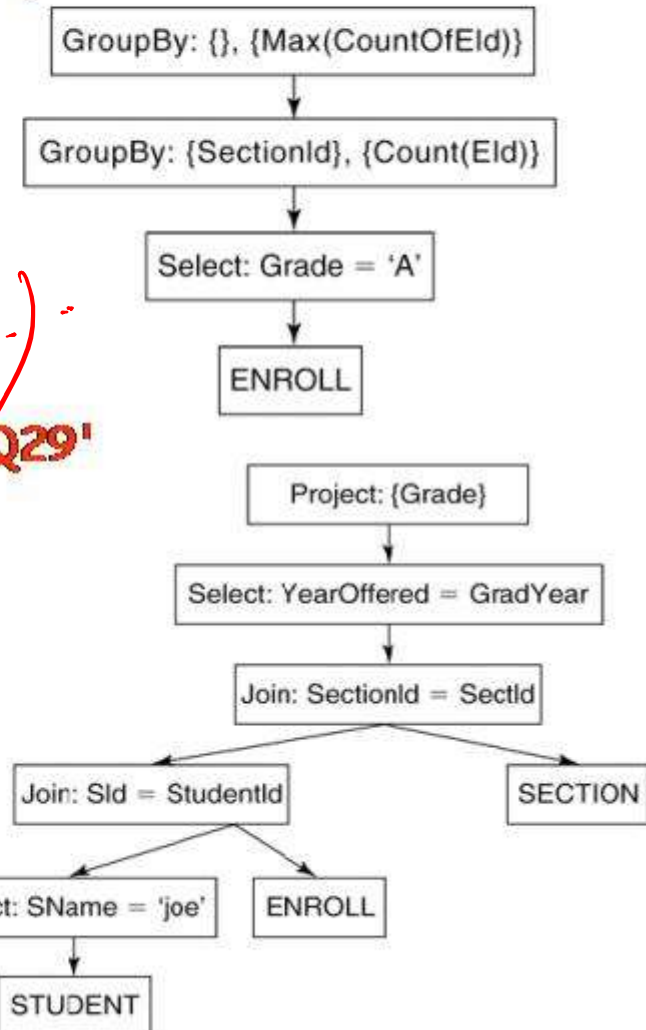
Q3:



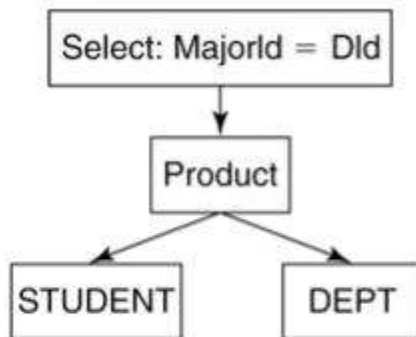
Q6:



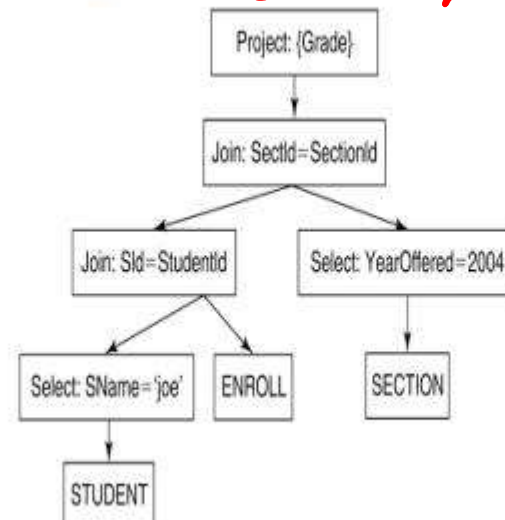
Q21:



Q23:



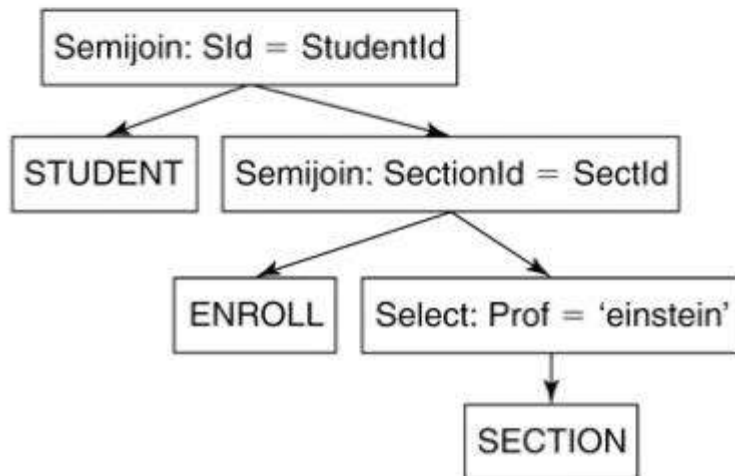
Q29:



Q29'

# Query Trees of previous examples

**Q40**



**Q51**

